

고속 인터페이스 · PHY · 신호무결성(SI/PI) 완전 이해

🔥 메모리의 대역폭 경쟁은 결국 "더 빠른 신호를 더 깨끗하게 보내는 싸움"이다. DRAM·HBM이 PHY와 인터페이스 설계를 핵심으로 두는 이유, 그리고 그 안에서 설계자가 끝까지 지켜내려는 단 하나의 단어 — **마진 (margin)** — 을 1st-principles로 끝까지 풀어낸다. 🧠 지원자 좌표: 로직 STA·CDC·검증·sign-off 4년. setup/hold 마진, jitter, corner, timing closure를 매일 다뤘다면 — 고속 I/O 마진은 "이미 알던 문제가 시간축 위에서 아날로그 옷을 입은 것"일 뿐이다. 이 문서는 그 옷을 벗기고 뼈대를 보여준다.

0. 이 문서를 읽는 법

이 문서는 암기용 요약집이 아니다. 정독용 해설서다. SK하이닉스 설계 JD에 반복적으로 등장하는 키워드 — "High Speed Interface/PHY", "RX/TX/ZQ/ODT/Equalizer/DCC", "SerDes/CDR/Jitter/BER", "Equalization(CTLE/DFE/FFE)", "PAM3/4", "SI/PI/EMC: reflection/crosstalk/PDN/IR-drop/decap", "TSV PHY/3D PHY", "DLL/PLL" — 이 키워드들은 면접에서 따로따로 묻지만, 실제로는 **하나의 사슬**로 엮여 있다.

그 사슬의 첫 고리는 이것이다: **신호는 빛의 속도로 가지 않는다**. 채널을 지나며 늦고, 번지고, 반사되고, 옆 신호에 오염되고, 클럭은 떨린다. 이 모든 열화를 정량화한 그림이 **아이 다이어그램(eye diagram)**이고, 아이 안에 남은 깨끗한 공간이 곧 **마진**이다. PHY 설계의 모든 블록 — 드라이버, 종단, ZQ, 등화기, DLL/PLL, DCC — 은 결국 "아이를 다시 크게 벌리는" 한 가지 목적을 위해 존재한다.

읽는 순서는 §1(왜) → §2(왜 깨지나) → §3~§6(어떻게 살리나: TX/RX/종단 → 등화 → 클럭회복 → 직병렬) → §7~§8(SI/PI) → §9(HBM 맥락) → §10(면접) → §11(자가점검)이다. 각 절은 앞 절의 "왜?"에 답하며 쌓인다.

1. 도입: 메모리 대역폭 경쟁의 본질

1.1 "더 빠른 신호를 더 깨끗하게"

메모리 산업의 모든 세대 전환은 결국 한 숫자를 키우는 싸움이다 — **대역폭(bandwidth)**. `B_hbm.md` 에서 본 식을 다시 보자.

$$\text{대역폭(BW)} = \text{I/O 폭(pin 수)} \times \text{per-pin 속도(Gbps)} \div 8$$

이 식에는 두 개의 손잡이밖에 없다. **폭을 넓히거나(병렬도↑), 핀당 속도를 올리거나(주파수↑)**. 그런데 둘 다 공짜가 아니다.

- 폭을 넓히면 → 핀·배선·면적·전력이 늘고, 수천 가닥이 동시에 스위칭하며 **전력무결성(PI)** 문제가 폭증한다.
- 속도를 올리면 → 같은 채널(배선)에서 신호가 깨지기 시작한다. 채널의 대역 한계에 부딪히고, **신호무결성(SI)** 문제가 폭증한다.

즉 메모리 대역폭 경쟁의 본질은 단순히 "빠르게"가 아니라 "빠르게 보내되, 받는 쪽에서 0과 1을 틀림없이 구분할 만큼 깨끗하게" 이다. 빠름과 깨끗함은 근본적으로 충돌한다. 이 충돌을 푸는 학문이 바로 **고속 인터페이스 / PHY / SI·PI** 설계다.

1.2 PHY란 무엇인가 — 디지털과 물리세계의 국경

칩 내부 코어 로직은 깨끗한 디지털 세계에 산다. 0은 0V, 1은 Vdd, 에지는 거의 수직. 그런데 이 신호를 칩 밖으로 내보내 인터포저·PCB·소켓을 지나 상대 칩까지 보내려면, **물리적 채널이라는 더러운 세계**를 통과해야 한다.

PHY(Physical Layer)는 바로 이 국경에 서 있는 회로 블록이다. 코어의 병렬 디지털 데이터를 받아 → 직렬화하고 → 채널 손실을 미리 보상(pre-emphasis/equalization)하고 → 드라이버로 밀어내고(TX), 반대로 들어오는 더러워진 신호를 받아 → 등화로 복원하고 → 클럭을 회복하고 → 병렬 디지털로 되돌려 코어에 넘긴다(RX). DRAM/HBM에서 "PHY"라 하면 보통 이 송수신 회로 + 종단(ODT) + ZQ 캘리브레이션 + DLL/DCC 같은 타이밍 블록 전체를 묶어 부른다.

DRAM이 PHY를 핵심으로 두는 이유는 명확하다. **셀 어레이가 아무리 빨리 데이터를 토해내도, PHY가 그것을 깨끗하게 밖으로 내보내지 못하면 대역폭은 0이다.** 03 A5 에서 정리했듯, 메모리 설계에서 디지털 RTL·타이밍·CDC가 사는 곳은 셀이 아니라 **Peri(주변회로)**이고, 고속 I/O PHY는 그 Peri의 가장 timing-critical한 심장이다.

1.3 설계자가 보는 "마진"이라는 단어

STA를 4년 한 사람에게 가장 익숙한 단어는 **slack**이다. E_sta.md 의 정의 그대로:

slack = required time - arrival time. (+ = 여유, - = 위반)

고속 I/O 세계의 "마진"은 이 slack의 **물리적·시간적·전압적 확장판**이다. 디지털 STA가 보던 setup/hold 마진은 "데이터가 클럭 에지 기준 안전 윈도우 안에 들어왔는가"였다. 고속 I/O에서는 이 윈도우가 두 축으로 확장된다.

- **시간 마진(timing margin)** = 아이 다이어그램의 가로 폭. 데이터 전이(transition)의 좌우 흔들림(jitter)이 클수록 좁아진다. ↔ STA의 setup/hold 윈도우.
- **전압 마진(voltage margin)** = 아이 다이어그램의 세로 높이. 노이즈·손실·반사로 0/1 레벨이 위아래로 번질수록 좁아진다. ↔ STA에는 없던 새 축(아날로그).

핵심 통찰: **고속 I/O 설계는 "STA 마진 사고에 전압축 하나를 더한 것"**이다. setup/hold가 시간축이라면, eye height가 전압축이다. 둘 다 "안전 윈도우를 얼마나 확보했나"라는 동일한 질문이다. 그래서 STA·timing closure 경험은 고속 I/O 마진 sign-off로 거의 1:1 이식된다(§10에서 이 브릿지를 완성한다).

2. 디지털 신호가 고속에서 깨지는 이유

이 절이 전체 문서의 물리적 토대다. "왜 빠르면 깨지나"를 끝까지 파고든다.

2.1 채널은 이상적 전선이 아니다 — RLC와 대역 제한

이상적 전선이라면 신호는 들어간 모양 그대로 나온다. 그러나 실제 배선(PCB trace, 인터포저 metal, TSV, 패키지 substrate)은 **저항(R), 인덕턴스(L), 커패시턴스(C)**를 가진 분포 회로다. 이 RLC가 만드는 결과:

1. **주파수에 따라 손실이 다르다 (대역 제한).** 고주파 성분일수록 더 크게 감쇠한다(skin effect로 $R \uparrow$, dielectric loss로 절연체가 에너지 흡수). 디지털 펄스는 푸리에로 보면 수많은 고조파의 합인데, 그 고조파들이 차별적으로 깎이면 날카롭던 에지가 둥글게 뭉개진다(rise/fall time $\uparrow$).
2. **신호가 늦는다 (지연·위상).** 채널은 일종의 저역통과필터(LPF)다. 비트 주기가 길 때(저속)는 한 비트가 충분히 정착(settle)한 뒤 다음 비트가 온다. 그러나 비트 주기가 짧아지면(고속) 앞 비트가 채 가라앉기도 전에 다음 비트가 도착한다.

여기서 결정적 현상이 나온다.

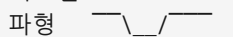
2.2 ISI — 심볼간 간섭, 가장 본질적인 적

ISI(Inter-Symbol Interference, 심볼간 간섭)는 고속 디지털의 1번 적이다.

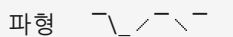
직관: 채널이 LPF이므로, 한 비트의 "꼬리(tail)"가 채널의 시정수만큼 끌린다. 비트 주기가 그 꼬리보다 짧으면, **이전 비트의 잔향이 현재 비트 위에 겹쳐진다**. 마치 잔향이 큰 강당에서 빠르게 말하면 앞 단어가 뒷 단어에 묻혀 알아들을 수 없는 것과 같다.

수치 예시(개념):

저속(비트 주기 $\gg$ 채널 시정수):

비트열 1 0 1 1
파형  ← 각 비트가 충분히 정착, 깨끗

고속(비트 주기 $\sim$ 채널 시정수):

비트열 1 0 1 1
파형  ← 0이 1 사이에서 완전히 안 내려감(잔향),
1과 1 경계 모호 → 판정 어려움 = ISI

ISI가 무서운 이유: **데이터 패턴 의존적**이다. $\dots 0\ 0\ 0\ 1\dots$ (긴 0 뒤 1)은 채널이 충분히 0으로 가라앉았다가 1로 가므로 전이가 늦고, $\dots 1\ 0\ 1\dots$ (빠른 토글)은 잔향이 누적된다. 즉 같은 비트라도 앞 패턴에 따라 도착 시각·레벨이 달라진다 → 아이 다이어그램에서 **여러 전이 궤적이 겹치며 아이를 안에서 갹아먹는다**. ISI를 보상하는 것이 §4의 등화(equalization)의 존재 이유다.

STA 브릿지: ISI는 본질적으로 "데이터 의존적 지연 변동"이다. STA에서 OCV/derating으로 path delay의 worst를 보던 것처럼, ISI는 "최악 패턴에서의 eye closure"를 본다. 다른 점은 STA가 셀-to-셀이라면 여기는 채널-to-eye라는 것뿐.

2.3 반사(reflection) — 임피던스 부정합의 대가

전송선(transmission line)에서 신호는 파동이다. 파동이 **임피던스가 다른 경계**를 만나면 일부가 반사된다 (연못에서 벽에 부딪힌 물결처럼). 채널 특성 임피던스가 Z_0 (보통 50 Ω 싱글엔드, 100 Ω 차동 — 변동)인데 송신단/수신단/커넥터/비아의 임피던스가 다르면, 그 불연속점마다 신호가 되튐다.

반사 계수(개념): $\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$. 완벽히 정합($Z_L = Z_0$)이면 $\Gamma=0$, 반사 없음. 개방($Z_L=\infty$)이면 $\Gamma=+1$ (전부 반사), 단락이면 $\Gamma=-1$.

되틴 신호는 채널을 왕복하며 원 신호에 더해진다 → **ringing(공진성 떨림)**, **overshoot/undershoot**, **ISI 악화**. 비트 주기가 왕복 시간과 비슷해지면 반사파가 다음 비트 판정 순간에 도착해 직접 오염시킨다.

대응은 단 하나의 원리: **임피던스를 맞춰라(termination)**. 이것이 §3의 ODT/종단이 존재하는 이유다.

2.4 Crosstalk — 옆 신호의 침범

배선은 혼자 가지 않는다. 수십~수천 가닥이 나란히 간다. 한 선(aggessor)이 스위칭하면, 그 전자기장이 **용량성(C)·유도성(L) 결합**으로 옆 선(victim)에 노이즈를 유도한다 — 이것이 **crosstalk**.

- **near-end crosstalk(NEXT)**: 송신단 쪽으로 결합.
- **far-end crosstalk(FEXT)**: 수신단 쪽으로 결합(고속 데이터 판정에 더 치명적).

crosstalk는 victim의 아이를 위아래로 흔들며 **전압 마진(eye height)**을 갉아먹는다. 특히 HBM처럼 수천 가닥이 좁은 인터포저에 밀집하면 crosstalk가 폭증한다(§9). 대응: 라우팅 간격↑, 가드 트레이스/접지 차폐, **차동 신호**(공통모드 결합 상쇄), 인접 신호 패턴 제어.

2.5 Jitter — 시간축의 떨림

지금까지(ISI·반사·crosstalk)가 주로 **전압 마진(세로)**을 갉았다면, **jitter**는 **시간 마진(가로)**을 갉는다.

E_sta.md 에서 본 정의를 그대로 가져온다: jitter는 클럭/데이터 에지가 이상적 위치에서 **시간적으로 흔들리는 변동**이다.

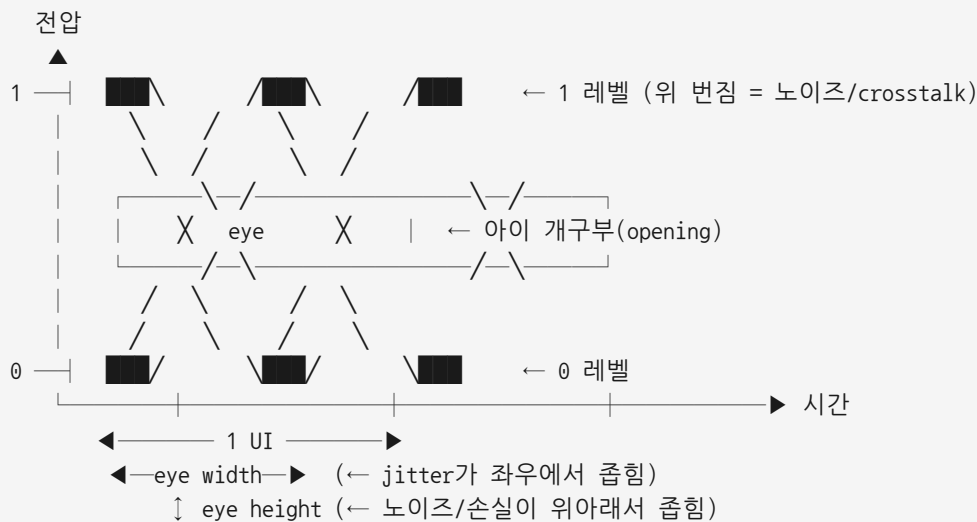
jitter의 분해(설계자가 알아야 할 수준): - **RJ(Random Jitter)**: 열잡음 등 무작위, 가우시안. 절대 0이 안 되며 BER 목표에 따라 통계적으로 확장된다(예: BER $1e-12$ → 약 $\pm 7\sigma$ — 변동). - **DJ(Deterministic Jitter)**: 원인이 명확한 jitter. 그 안에 다시 - **ISI 유발 jitter (DDJ, data-dependent)**: §2.2의 패턴 의존 전이 변동. - **PJ(Periodic Jitter)**: 전원 노이즈·간섭 등 주기적. - **DCD(Duty Cycle Distortion)**: 듀티 사이클 왜곡 → §5의 DCC가 보상.

총 jitter $\approx$ DJ + (RJ를 BER만큼 확장). 이 총 jitter가 아이의 좌우 폭을 좁힌다.

STA 브릿지: 이것은 E_sta.md Q3의 $uncertainty = skew + jitter + margin$ 을 시간축에서 더 정밀하게 분해한 것이다. STA에서 jitter를 한 숫자(clock uncertainty)로 보던 것을, 고속 I/O에서는 RJ/DJ/DCD로 나눠 각각 다른 회로(PLL/DCC/등화)로 잡는다. "jitter가 누가 만드나?" — E_sta.md Q3 꼬리질문의 답(PLL/DLL, 전원 노이즈, crosstalk)이 바로 여기서 P1·S1와 연결된다.

2.6 아이 다이어그램 — 마진을 한 장으로 보는 법

지금까지의 모든 열화를 한 장의 그림으로 종합한 것이 아이 다이어그램이다. 만드는 법: 수신 신호를 1 UI(Unit Interval, 1비트 주기)마다 잘라 **모두 겹쳐 그린다**. 수많은 비트의 궤적이 겹치면, 가운데에 눈(eye) 모양의 빈 공간이 생긴다.



- 아이 가로 폭(**eye width**) = 시간 마진 = 1 UI - 총 jitter. 클수록 클럭 회복·샘플링 여유.
- 아이 세로 높이(**eye height**) = 전압 마진 = 판정 임계에서 0/1 레벨까지 여유. 클수록 노이즈 견딤.
- 아이가 닫힌다(**eye closure**) = 마진 소실 = 비트 에러. 완전히 닫히면 BER↑↑.

수신기는 아이의 **정중앙(시간·전압 모두 가장 여유 있는 점)**에서 샘플링하려 한다. 그 중앙을 찾아주는 것이 §5의 CDR/DLL이고, 닫힌 아이를 다시 벌리는 것이 §4의 등화다.

한 문장 요약: 아이 다이어그램은 "STA의 setup/hold 윈도우 + 전압 노이즈 마진"을 한 그림에 합친 것이며, 고속 I/O 설계의 모든 블록은 이 아이를 크게 유지하려는 노력이다.

3. 송수신 기본 블록: TX · RX · 종단 · ZQ

아이를 살리는 회로들을 본다. 먼저 가장 기본 골격.

3.1 TX(드라이버) — 신호를 채널에 밀어넣는 힘

TX 드라이버는 코어의 디지털 비트를 받아 채널 임피던스에 맞춰 전류/전압으로 변환해 밀어낸다. 핵심 요구사항: - **임피던스 정합:** 드라이버 출력 임피던스 $\approx$ 채널 Z_0 (보통 $\sim 34\Omega$ 또는 40Ω 클래스 — 변동). 불일치 시 반사 (§2.3). - **slew rate 제어:** 너무 빠른 에지는 고주파 $\uparrow \rightarrow$ EMI·crosstalk $\uparrow$. 너무 느리면 ISI $\uparrow$. 절충. - **드라이브 세기 조절:** 채널 길이·부하에 맞춰 출력 강도 가변(impedance/strength code). - **pre-emphasis/FFE:** 보낼 때 미리 고주파를 강조해 채널 손실을 선보상 (§4.1).

DRAM에서 TX는 출력 드라이버(off-chip driver, OCD)로 불리며, 그 출력 임피던스를 정확히 맞추기 위해 **ZQ 캘리브레이션** (§3.4)이 필요하다.

3.2 RX(리시버) — 더러워진 신호에서 0/1을 건지는 회로

RX는 채널을 지나며 감쇠·왜곡된 미약한 신호를 받아 디지털로 복원한다. 핵심: - **비교(slicing):** 입력을 기준 전압 (V_{ref})과 비교해 0/1 판정. 차동이면 두 선의 차로 판정(공통모드 노이즈 제거). - **민감도/대역:** 작은 아이도 판정할 수 있는 이득과 대역. - **등화:** CTLE(아날로그)·DFE(판정 피드백)로 채널 손실·ISI 복원 (§4). - **종단(ODT):** 입력단 반사 흡수 (§3.3). - **V_{ref} 트레이닝:** 판정 임계를 아이 중앙 높이에 맞춤(DDR4부터 V_{ref} training).

메모리 브릿지(03 A4): DRAM 셀의 sense amp가 BL의 수십 mV 차이를 증폭해 0/1로 가르는 것과, I/O RX가 작은 아이를 Vref와 비교해 0/1로 가르는 것은 같은 "미약 신호 판정 + 마진" 문제다. sense margin = eye height의 셀 어레이판.

3.3 종단(ODT/termination) — 반사를 흡수하는 저항

반사(§2.3)를 막는 직접적 방법은 채널 끝(또는 양 끝)에 Z_0 와 같은 저항을 달아 남은 에너지를 흡수하는 것이다. 이것이 **termination**, DRAM에서는 칩 안에 내장한 **ODT(On-Die Termination)**.

두 가지 큰 분류:

구분	series termination(직렬)	parallel termination(병렬)
위치	TX 출력에 직렬 저항	RX 입력에서 전원/접지로 병렬 저항
원리	드라이버 임피던스 + 직렬 $R = Z_0 \rightarrow$ 반사파를 source에서 재흡수	라인 끝 임피던스를 Z_0 로 맞춰 처음부터 반사 안 생김
정적 전력	낮음(DC 전류 거의 없음)	높음(종단 저항에 상시 전류)
신호 품질	점대점·저속에 유리	고속·멀티드롭에 유리(반사 즉시 제거)
비유	"보낸 메아리를 보낸 사람이 흡수"	"받는 벽에 흡음재를 붙임"

DDR/HBM 같은 고속 인터페이스는 주로 **parallel ODT(또는 둘의 조합)**를 쓰며, ODT 값을 동작 조건에 따라 동적으로 켜고 끈다(읽기/쓰기, 어느 rank가 active인가에 따라). 멀티랭크 DDR에서 "어느 칩이 ODT를 켜는가"는 SI 최적화의 핵심 변수다.

STA/CDC 브릿지: ODT의 켜고 끄는 **command/timing에 종속된 제어 로직**이다. 언제 ODT를 enable할지의 타이밍이 틀리면 그 순간 반사로 아이가 닫힌다 $\rightarrow$ 이 ODT 제어 FSM·타이밍은 정확히 Peri 디지털 로직의 STA·검증 대상이다.

3.4 ZQ 캘리브레이션 — 임피던스를 "현장에서" 보정하는 이유

여기서 1st-principles 질문: 왜 드라이버/ODT 임피던스를 설계 때 고정값으로 못 박지 못하고, 실행 중에 보정(calibration)해야 하나?

답: **PVT 변동**. 온칩 저항·트랜지스터의 실제 값은 공정(Process) 산포로 $\pm$ 수십%까지 흔들리고, 동작 중 전압(V)·온도(T)에 따라 또 변한다. E_sta.md에서 STA가 PVT corner를 보는 것과 같은 변동이, 여기서는 **출력 임피던스의 변동**으로 나타난다. 임피던스가 변하면 정합이 깨지고(§2.3 반사), ODT 값도 틀어진다.

ZQ 캘리브레이션의 원리: 1. 보드에 정밀 외부 기준 저항 R_{ZQ} (예: 240Ω — 변동)를 ZQ 핀에 단다. 이 저항은 PVT에 안 변하는 정밀 부품. 2. 칩 내부 드라이버/ODT의 가변 임피던스(여러 트랜지스터 leg를 켜고 끄)를 이 외부 기준과 비교한다. 3. 비교 결과로 "몇 개의 leg를 켜야 목표 임피던스가 되는가"의 **코드(pull-up/pull-down code)**를 찾는다. 4. 주기적으로(온도가 변하므로) 재보정(ZQCS/ZQCL 명령).

즉 ZQ는 "PVT로 흔들리는 온칩 임피던스를, 안 흔들리는 외부 기준에 맞춰 디지털 코드로 정렬"하는 과정이다.

강력한 브릿지: ZQ 캘리브레이션은 본질적으로 "아날로그 변동을 디지털 보정 코드로 잡는 피드백 루프"다. STA에서 PVT corner를 derating으로 다루던 사고가, 여기서는 corner를 런타임에 calibration으로 추적하는 것으로 진화한다. 그리고 이 보정 루프(비교기 + FSM + 코드 레지스터)는 디지털 제어 로직 — 설계·검증 대상.

4. 등화(Equalization): 채널 손실을 되돌리는 기술

§2에서 ISI는 "채널이 LPF라 고주파가 깎여 생긴다"고 했다. 그렇다면 해법은 자명하다: **깎인 고주파를 도로 살린다 (또는 미리 강조해 보낸다)**. 이것이 등화(equalization)이고, DDR5·HBM·GDDR·모든 SerDes의 핵심 기술이다.

세 가지 주력 기법을 위치(TX/RX)와 방식(아날로그/디지털)으로 나눠 본다.

4.1 FFE (Feed-Forward Equalizer, TX 측) — 미리 강조해 보낸다

FFE(또는 pre-emphasis/de-emphasis)는 송신 쪽에서 **채널이 깎을 것을 미리 알고 보상**하는 기법이다. 직관: 채널이 고주파를 깎을 테니, 보낼 때 전이(transition) 비트는 세게, 같은 값이 이어지는 비트는 약하게 보낸다. 그러면 채널을 지난 뒤 평평해진다.

구현: 현재 비트뿐 아니라 인접 비트(탭, tap)의 가중합으로 출력 파형을 만든다(FIR 필터와 동형).

TX out = $w(-1) \cdot b(n-1) + w(0) \cdot b(n) + w(+1) \cdot b(n+1) \dots$ (탭 가중치 w)

- **장점:** 디지털로 정밀 제어, 안정적(피드백 없음).
- **단점:** 보낼 때 고주파를 키우는 게 아니라 **저주파를 깎는 효과(de-emphasis)** → 전체 신호 진폭(eye height)이 줄어든다. crosstalk을 같이 증폭할 수도.

4.2 CTLE (Continuous-Time Linear Equalizer, RX 아날로그) — 받자마자 고역 부스트

CTLE는 수신단 맨 앞에 두는 **아날로그 고역통과 부스트 필터**다. 채널이 LPF로 고주파를 깎았으니, CTLE는 그 역특성(고주파 이득↑)을 줘서 평탄화한다.

- **장점:** 전력·면적 작고, 넓은 대역을 연속적으로 보상. RX 맨 앞에서 작은 신호를 먼저 키워 후단 부담↓.
- **단점:** **신호도 노이즈·crosstalk도 같이 증폭**한다(선형이라 차별 못함). 채널 손실이 너무 크면 CTLE만으로 부족.

비유: CTLE는 "전체 볼륨에서 고음만 키우는 이퀄라이저 노브" — 음악도 잡음도 같이 커진다.

4.3 DFE (Decision-Feedback Equalizer, RX) — 이미 판정한 비트로 잔향을 뺀다

DFE가 가장 영리하다. 핵심 아이디어: ISI의 정체는 "**이전 비트들의 잔향**"이다(§2.2). 그런데 이전 비트들은 이미 0/1로 판정했으니, 그 값을 안다. 안다면, 그 잔향이 현재 비트에 더할 양을 계산해서 빼버리면 된다.

판정값 $d(n) = \text{slice}(\text{입력}(n) - \sum c(k) \cdot d(n-k))$ (이미 판정한 과거 비트 $d(n-k)$ 로 보정)
 $\uparrow$ 피드백: 과거 판정 $\times$ 탭계수

- **결정적 장점:** 과거 비트는 이미 "깨끗한 0/1"이므로, 노이즈를 증폭하지 않고 ISI만 정확히 제거한다. CTLE/FFE가 노이즈도 키우는 것과 근본적으로 다르다.
- **단점:** (1) 피드백 루프라 1 UI 안에 판정→보정이 끝나야 한다(고속에서 타이밍 빡빡, critical path). (2) 첫 탭(가장 가까운 과거)이 가장 중요한데 그 타이밍이 가장 어렵다. (3) 판정이 틀리면 그 오류가 피드백으로 전파(error propagation).

4.4 왜 DDR5는 DFE를 넣었나 — 그리고 셋의 조합

DDR4까지는 속도가 낮아(상대적으로) 채널 손실·ISI가 견딜 만했다. 그러나 DDR5(3.2~6.4 Gbps급 — 변동)로 가며 채널 손실·ISI가 아이를 달기 시작했고, 특히 **read 경로의 ISI**가 문제가 됐다. 그래서 DDR5는 **DRAM RX(컨트롤러→DRAM 쓰기 경로 등)에 DFE를 도입했다**. DFE를 고른 이유는 §4.3의 결정적 장점 — **노이즈를 키우지 않고 ISI만 제거**하기 때문이다. DDR 메모리는 채널이 멀티드롭·반사로 ISI가 지배적이라, 노이즈 증폭형(CTLE)보다 ISI 표적 제거형(DFE)이 효율적이다.

실제 고속 링크는 셋을 조합한다:

TX(FFE 선보상) — 더러운 채널 —▶ RX[CTLE(아날로그 평탄화) → DFE(ISI 정밀 제거) → slicer]

- FFE가 보내기 전 거칠게 보상 → CTLE가 받자마자 고역 복원 → DFE가 남은 ISI를 비트 단위로 정밀 제거.

기법	위치	방식	노이즈 증폭	핵심 한계
FFE	TX	디지털 FIR	간접(de-emphasis로 진폭↓)	eye height 희생
CTLE	RX	아날로그	예(선형이라 같이 증폭)	손실 큰 채널엔 부족
DFE	RX	디지털 피드백	아니오(과거 판정 사용)	루프 타이밍·error propagation

STA 브릿지: DFE의 피드백 루프는 "1 UI 안에 판정→감산→다음 판정"이 닫혀야 하는 **timing-critical loop**다. 이것은 STA의 $\text{setup: } T_{\text{clk}} + \text{skew} \geq t_{\text{cq}} + t_{\text{comb}} + t_{\text{su}} (E_{\text{sta.md}} Q1)$ 와 정확히 같은 타이밍 단힘 문제다 — 다만 T_{clk} 자리에 1 UI가, comb path에 감산기+슬라이서가 들어갈 뿐. DFE 탭의 타이밍 클로저는 STA 사고 그대로다.

5. 클럭/타이밍 회복: 소스동기 · DLL/PLL · DCC · CDR

데이터를 깨끗이 만들어도(§3·§4), **언제 샘플링할지(클럭)**가 틀리면 소용없다. 아이의 정중앙을 어떻게 찾아 샘플링하느냐 — 이것이 타이밍 회복이다.

5.1 두 갈래: 소스 동기(source-synchronous) vs 임베디드 클럭

소스 동기 방식: 데이터를 보내는 쪽이 클럭(또는 스트로브)을 데이터와 함께 같이 보낸다. 수신단은 그 클럭으로 데이터를 받는다. → DDR/HBM이 이 방식. DDR의 DQS(Data Strobe)가 바로 이 동행 클럭이다. 데이터(DQ)와

스트로브(DQS)가 같은 채널을 같이 지나므로 jitter·지연이 공통(common)으로 상쇄되어 정렬이 쉽다.

임베디드 클럭 방식: 별도 클럭을 안 보내고, 데이터 천이 자체에서 클럭을 복원한다. → SerDes/PCIe/이더넷이 이 방식. 핀을 아끼지만, 수신단이 데이터로부터 클럭을 뽑아내는 CDR(§5.4)이 필요하다. (긴 동일 비트열에서 천이가 없으면 클럭을 못 뽑으니 8b/10b 등 인코딩으로 천이를 보장한다.)

메모리는 대부분 소스 동기(DQS)다. 그래서 DRAM PHY 타이밍의 핵심은 "DQ를 DQS의 어느 위치로 정렬하느냐" — 즉 read/write leveling과 training이다(E_sta.md Q1 브릿지의 "DQS로 DQ를 캡처"가 정확히 이것).

5.2 DLL vs PLL — 메모리에서 왜, 어떻게 다른가 (★ 빈출)

03 c9 의 핵심 질문이다. 둘 다 클럭을 다루는 피드백 회로지만 목적과 원리가 다르다.

PLL(Phase-Locked Loop, 위상고정루프): - 원리: 위상 검출기 → 루프 필터 → VCO(전압제어 발진기) → 분주기 → 피드백. 입력 클럭에 위상·주파수를 잠근다. - 핵심 능력: VCO가 있어 주파수 합성(체배/분주)이 가능. 입력 100MHz로 1GHz를 만들 수 있다. - 단점: VCO 자체가 jitter를 만들 수 있고, 면적·전력·락 시간이 크다.

DLL(Delay-Locked Loop, 지연고정루프): - 원리: 위상 검출기 → 제어 → 가변 지연선(delay line) → 피드백. VCO 대신 지연 사슬을 조절해 위상을 맞춘다. - 핵심 능력: 위상 정렬(skew 제거)에 특화. 발진기가 없어 jitter 누적이 없고(입력 jitter를 증폭 안 함), 구조가 단순·안정. - 단점: 발진기가 없어 주파수 체배는 못한다(위상만 맞춤).

메모리에서의 쓰임(직관): - DRAM은 외부에서 들어온 클럭으로 데이터를 내보내는데, 내부 경로 지연 때문에 출력 DQ/DQS가 외부 클럭 대비 들어진다. 이 내부 지연을 보상해 출력 클럭을 외부 클럭에 정렬하는 데 DLL이 안성맞춤이다(주파수는 그대로, 위상만 맞춤). → 전통적으로 DDR DRAM은 출력 타이밍 정렬에 DLL을 썼다. - 반면 주파수 합성·체배가 필요한 곳(고속 직렬 클럭 생성 등)에는 PLL이 쓰인다. HBM·고속 PHY는 둘을 상황에 맞게 혼용한다.

항목	PLL	DLL
핵심 소자	VCO(발진기)	delay line(지연선)
잘하는 일	주파수 합성(체배/분주)	위상 정렬(skew 제거)
jitter	VCO가 누적·생성 가능	입력 jitter 증폭 안 함(누적 없음)
주파수 변경	가능	불가(위상만)
메모리 전형 용도	내부 고속 클럭 생성	출력 DQ/DQS를 외부 클럭에 정렬

STA 브릿지: E_sta.md Q3 꼬리질문 "jitter는 누가 만드나? → PLL/DLL". 여기서 그 답을 회로 수준으로 본 것이다. PLL은 VCO 때문에 jitter를 만들 수 있고 DLL은 안 만든다 — 이 차이가 곧 고속 I/O 마진(eye width)에 직접 영향. STA에서 clock uncertainty로 한 줄 쓰던 jitter의 출처를 회로로 추적하면 PLL/DLL 선택 문제가 된다.

5.3 DCC (Duty Cycle Corrector) — 듀티 50%를 지키는 이유

DDR은 클럭의 상승·하강 양 에지에서 데이터를 받는다(Double Data Rate). 따라서 클럭이 정확히 듀티 50% (high 구간 = low 구간)여야 두 에지의 데이터 윈도우가 같다. 그런데 채널·드라이버 비대칭·온도로 듀티가 틀어지면(DCD, §2.5), 한쪽 에지의 아이가 좁아진다 → 절반의 비트가 마진을 잃는다.

DCC(Duty Cycle Corrector)는 클럭의 high/low 비율을 측정해 50%로 되돌리는 보정 회로다. 고속 DDR/HBM일수록 1 UI가 짧아 작은 듀티 오차도 치명적이라 DCC가 필수가 된다.

STA 브릿지: 듀티 왜곡은 STA에서 보던 clock의 high/low pulse width 위반과 동형이다. min pulse width check가 깨지면 안 되듯, 고속 I/O에서 듀티 50%가 깨지면 한쪽 에지 샘플링이 무너진다. DCC는 그 pulse width를 런타임에 능동 보정하는 회로.

5.4 CDR (Clock Data Recovery) — 데이터에서 클럭을 뽑는다

SerDes(임베디드 클럭, §5.1)에서, 수신단은 별도 클럭이 없으니 데이터 천이의 위치로부터 클럭을 복원해야 한다 — CDR. 원리: 들어오는 데이터 에지와 내부 회복 클럭의 위상을 비교(위상 검출) → 루프로 클럭 위상을 데이터 천이에 맞춰 잠근다 → 그 클럭의 위상을 데이터 사이 아이 정중앙으로 정렬해 샘플링.

- 천이가 있어야 위상을 비교하니 천이 보장 인코딩(8b/10b, scrambling)이 필요.
- CDR이 데이터 jitter를 얼마나 따라가느냐(추종 대역)와, 자체가 만드는 jitter가 마진을 좌우.

5.5 Jitter와 BER의 관계 — 왜 결국 "확률"인가

마지막 1st-principles: 왜 고속 링크는 0/1을 "확실히"가 아니라 "확률적으로" 받나? jitter(특히 RJ)는 가우시안이라 꼬리가 무한하다. 즉 아무리 작아도 에지가 아이 가장자리를 넘어 샘플링 순간을 오염시킬 확률이 0이 아니다. 그 확률이 곧 BER(Bit Error Rate).

$BER \approx P(\text{아이 가장자리에서 jitter/노이즈가 판정점을 침범})$

- 아이가 클수록(eye width·height↑) → 침범 확률↓ → BER↓.
- 등화·DCC·낮은 jitter PLL·CDR은 결국 아이를 키워 BER 목표(예: $1e-15$ 급 — 변동)를 맞추는 일.
- bathtub curve: 샘플링 위치를 좌우로 옮기며 BER을 그리면 욕조 모양 — 바닥(낮은 BER)이 넓을수록 timing 마진이 크다.

STA 브릿지: STA는 "slack ≥ 0 이면 만족(결정론적)"이라 했다. 고속 I/O는 한 발 더 나가 "slack이 통계적으로 얼마의 확률로 음수가 되나"를 본다 — POCV/AOCV($E_{sta,md} Q5$)에서 통계적 타이밍을 보던 사고의 연장선이다. 결정론적 마진 → 통계적 마진(BER)으로 확장되는 것뿐, 질문은 동일하다.

6. 직렬화/병렬화: SerDes · 폭 vs 속도 · PAM4

6.1 SerDes — 왜 직렬화하나

SerDes(Serializer/Deserializer): TX에서 코어의 넓은 병렬 데이터(예: 32비트)를 한 가닥의 고속 직렬 스트림으로 묶어 보내고(serialize), RX에서 다시 병렬로 푼다(deserialize). 이유: 핀·배선 수를 줄이고, 적은 핀으로 높은 대역을 낸다. 대신 한 가닥의 속도가 극단으로 올라가 §2의 모든 문제(ISI·jitter·등화)가 심해진다.

6.2 폭 vs 속도 — GDDR(좁고 빠름) vs HBM(넓고 적당)

§1.1의 대역폭 식 $BW = \text{폭} \times \text{속도} \div 8$ 에서 같은 BW를 내는 두 철학:

	GDDR (좁고 빠름)	HBM (넓고 적당)
I/O 폭	좁음(예: 32-bit/칩급)	초광폭(1024~2048-bit)
per-pin 속도	매우 높음(GDDR7 PAM3 등, 20~30+ Gbps급 — 변동)	moderate(6.4~10 Gbps급 — 변동)
SI 난도	per-pin 고속 → ISI·등화 부담↑	per-pin 저속 → SI는 쉬움
배선/면적	적은 핀(보드 라우팅 유리)	막대한 핀 → 인터포저 필수, 라우팅·PI 부담↑
등화 필요	DFE/FFE 강하게 필요	상대적으로 덜
bit당 에너지	높음(고속 스위칭)	낮음(저속 광폭)

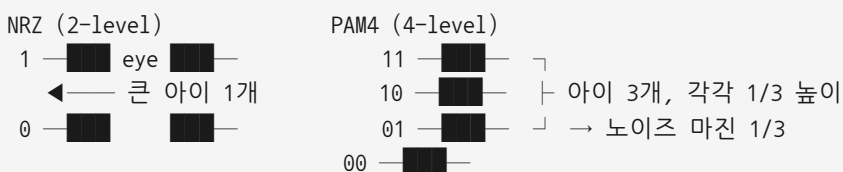
B_hbm.md Q1 브릿지 그대로: "폭을 넓혀 속도를 낮춘다"는 전형적 버스 설계 선택. GDDR은 협폭 고속이라 per-pin SI·등화에 사력을 다하고, HBM은 광폭 저속이라 per-pin SI는 쉽지만 수천 핀의 PI와 인터포저 라우팅·타이밍으로 난도가 옮겨간다(§8·§9).

6.3 PAM4/PAM3 — 한 심볼에 더 많은 비트, 그 대가

지금까지는 NRZ(2레벨, 1심볼=1비트)를 가정했다. 대역을 더 짜내려면 한 심볼에 더 많은 정보를 실으면 된다.

- PAM4(4-level): 전압을 4레벨로 나눠 1심볼 = 2비트. 같은 보율(baud)로 2배 비트레이트. → 채널 대역 부담은 그대로 두고 데이터율 2배.
- PAM3(3-level): 3레벨(GDDR7 채택), NRZ와 PAM4 사이 절충.

그러나 공짜가 아니다 — SNR 손해. NRZ는 0~Vdd 사이에 아이가 하나(2레벨). PAM4는 같은 진폭을 3개의 아이로 쪼갠다(4레벨 사이 3개의 눈). 각 아이의 세로 높이는 대략 1/3로 줄고, 판정 임계도 3개가 된다.



- 이득: 같은 채널 대역으로 2배 비트레이트(고주파 손실이 큰 채널에서 유리).
- 손해: eye height $\approx 1/3$ → SNR·노이즈 마진 약 1/3(약 -9.5dB — 변동). crosstalk·노이즈에 훨씬 취약 → 더 강한 등화(DFE)·FEC가 거의 필수. 판정 복잡(3 임계).

따라서 PAM4는 "채널 대역이 부족할 때(고주파 손실 지배적) 비트를 더 심는" 트레이드오프다. HBM은 광폭 저속이라 아직 NRZ가 주류이고, GDDR7은 PAM3로, 일부 초고속 SerDes는 PAM4로 간다.

STA 브릿지: NRZ→PAM4는 "타이밍 마진(가로)을 유지하되 전압 마진(세로)을 1/3로 깎아 데이터율 2배 심는" 선택이다. STA에서 voltage scaling이 timing 마진을 깎듯, PAM4는 voltage 마진을 깎는다 — 마진 예산의 재배분이라는 점에서 동일한 사고.

7. SI(신호무결성): 깨끗한 전송을 위한 대응

§2가 "왜 깨지나"였다면 §7은 "어떻게 막나"의 종합(03 B5 확장)이다. SI는 **신호 그 자체의 무결성** — 0이 0으로, 1이 1로, 제 시각에 도착하게 하는 것.

핵심 대응 5가지:

1. **임피던스 매칭 / termination(§3.3)** — 반사 제거. 채널 전 구간 Z_0 일관성 유지(비아-커넥터 불연속 최소화). ODT로 흡수.
2. **차동 신호(differential)** — 두 선의 차로 전송. 공통모드 노이즈(전원 노이즈·crosstalk)가 양 선에 같이 실리면 차에서 상쇄. 노이즈 내성↑, 클럭/스트로브·고속 링크에 광범위. (단 핀 2배.)
3. **length matching** — 한 버스의 모든 비트(또는 차동 쌍의 +/-)가 같은 시각에 도착하도록 배선 길이를 맞춤. 불일치 = skew = 아이 좌우 어긋남. (serpentine 라우팅으로 길이 보정.)
4. **라우팅 간격·차폐(spacing/shielding)** — crosstalk(§2.4) 억제. 신호 간 간격↑, 가드 트레이스·접지층으로 격리, aggressor와 victim 분리.
5. **에지/대역 관리** — slew rate 제어(너무 빠른 에지 = 고조파↑ = EMI·crosstalk↑), 등화로 ISI 제거(§4).

SI 문제	원인(§2)	주 대응	깎는 마진
reflection/ringing	임피던스 부정합	termination/매칭	전압+시간
crosstalk	인접선 결합	간격·차폐·차동	전압(eye height)
ISI	채널 대역 제한	등화(FFE/CTLE/DFE)	전압+시간
jitter	클럭/전원/crosstalk	저jitter PLL·등화	시간(eye width)

강점 브릿지: SI 대응은 전부 "**마진을 확보하라**"는 한 명제로 수렴한다. 03 B5 브릿지대로, SI/PI = 마진 확보 = 검증 마진 사고. STA에서 setup/hold 마진을 sign-off하던 것과 동일한 사고로 고속 I/O 마진을 본다.

8. PI(전력무결성): 전원이 흔들리면 신호도 흔들린다

SI만큼 중요한, 때로 더 중요한 짝이 **PI(Power Integrity, 전력무결성)**다. 직관: 회로는 깨끗한 전원(일정한 Vdd)을 가정하고 동작한다. 그런데 전원 자체가 출렁이면, 그 위에 실린 신호의 기준(0/1 레벨, Vref, 드라이버 세기)이 다 흔들린다.

8.1 PI를 깨는 세 현상

1. **IR drop**: 전류(I)가 전원망(PDN)의 저항(R)을 지나며 생기는 **전압 강하($V=IR$)**. 칩 멀리·깊은 곳일수록 Vdd가 낮아짐. 동작 전압이 떨어지면 셀 delay↑ → 타이밍 마진↓.
2. **SSN / di/dt noise(Simultaneous Switching Noise, 동시 스위칭 노이즈)**: 많은 출력이 **동시에** 스위칭하면 순간 전류 변화(di/dt)가 크고, 전원망 인덕턴스(L)에 $V = L \cdot di/dt$ 만큼의 **순간 전압 출렁임**이 생긴다. → 가장 무서운 PI 현상.
3. **ground bounce**: SSN의 접지판 버전. 동시 스위칭 전류가 접지 인덕턴스에 떨어뜨리는 전압으로 **기준 접지(0V)가 들썩임** → 모든 신호의 기준이 흔들림.

8.2 PDN과 decap — 어떻게 막나

PDN(Power Delivery Network, 전력 전달망): VRM(전원)에서 칩의 트랜지스터까지 전원을 나르는 전체 경로 (보드 평면 → 패키지 → 온칩 그리드). PI 설계의 목표는 **PDN의 임피던스를 넓은 주파수에서 낮게 유지**하는 것. PDN 임피던스가 낮을수록 전류 출렁임에도 전압이 덜 흔들린다($V = Z_{\{PDN\}} \cdot I$).

decap(decoupling capacitor, 디커플링 커패시터): 전원 근처에 둔 작은 전하 저수지. **순간 전류 수요를 국소적으로 즉시 공급**해, di/dt가 먼 VRM까지 전달되기 전에 흡수한다. 주파수대별로 다른 decap을 둔다(보드 bulk cap = 저주파, 패키지 cap = 중주파, **온칩 decap = 고주파/순간**). 비유: decap은 "수도 본관이 못 따라잡는 순간 수요를 받쳐주는 동네 물탱크."

8.3 왜 고속 광폭 I/O에서 PI가 SI만큼(혹은 더) 중요한가

1st-principles로: 고속이면 di/dt가 크고(빠른 스위칭), **광폭이면 동시에 스위칭하는 출력 수가 막대**하다. $SSN \propto (\text{스위칭 출력 수}) \times (di/dt)$. HBM처럼 **수천 핀이 동시에** 토글하면 SSN이 폭증한다. 그 전원 출렁임은: - 드라이버 출력 레벨을 흔들고(eye height↓), - 기준 전압(Vref/접지)을 흔들어 RX 판정을 어긋나게 하고, - **클럭·PLL/DLL을 흔들어 jitter를 만든다**(eye width↓).

즉 **PI 열화는 곧장 SI 열화로 변신**다. 그래서 광폭 고속 I/O에서는 "신호 라우팅만 잘해서는 안 되고, 전원이 흔들리지 않아야" 아이가 열린다. PI는 SI의 전제 조건이다.

STA/SI 통합 브릿지: E_sta.md Q3 "jitter는 누가 만드나 → 전원 노이즈"가 바로 PI다. 그리고 03 B6 브릿지의 "전원 노이즈가 timing에 영향"이 IR drop·SSN으로 구체화된다. STA의 voltage corner(저전압 = 느림)를 정적으로 보던 것을, PI는 동적 전원 출렁임으로 본다 — 같은 "전압이 타이밍에 미치는 영향" 문제의 정적/동적 두 얼굴.

8.4 EMC — 한 줄 보탬

EMC(Electromagnetic Compatibility): 빠른 에지·큰 di/dt는 전자기 방사(EMI)를 일으켜 자신·이웃을 방해한다. slew rate 제어, 차폐, 접지 설계, spread-spectrum 클럭킹 등으로 관리. SI·PI와 한 묶음으로 다뤄진다.

9. HBM / 2.5D 맥락: TSV PHY · 3D PHY · 초광폭의 폭증

이제 §1~§8을 HBM이라는 가장 극단적 무대에 올린다(B_hbm.md · 03 A8 · 03 E1 과 연결).

9.1 TSV PHY / 3D PHY — 수직으로 가는 인터페이스

B_hbm.md 대로 HBM은 DRAM die를 TSV(Through-Silicon Via)로 수직 적층하고, 최하단 **base(logic) die**가 외부 인터페이스를 담당한다. 여기서 PHY가 두 층위로 존재한다.

- **3D PHY (스택 내부, die→die):** TSV·micro-bump를 통한 수직 신호. 거리가 짧아(수십~수백 μm) per-link은 비교적 깨끗하지만, TSV의 기생 C-저항, micro-bump 접합 변동이 SI에 영향. 수천 개 TSV가 동시에 스위칭 → 내부 PI(SSN)도 심각.
- **2.5D 인터포저 PHY (base die ↔ GPU):** 실리콘 인터포저 위 미세 배선으로 GPU와 연결. 거리가 PCB보다 짧고 미세해 고밀도가 가능하지만, **2048-bit가 좁은 인터포저에 밀집** → crosstalk·라우팅·PI가 폭증.

9.2 2048-bit 초광폭에서 SI/PI/타이밍이 왜 폭증하나

03 A8 꼬리질문 "2048-bit 초광폭 I/O의 신호무결성"을 1st-principles로 전개한다. HBM4가 폭을 1024→2048로 2배 늘린 선택(B_hbm.md Q5)은 per-pin 속도 부담을 낮추는 현명한 수지만, **다른 곳에서 난도가 폭증한다.**

1. **PI(SSN) 폭증** — §8.3 그대로. 동시에 스위칭하는 핀이 2배 → $\text{SSN} \propto \text{핀 수}$ → 전원 출력임 폭증. base die의 PDN·온칩 decap 설계가 극한.
2. **crosstalk 폭증** — §2.4. 2048가닥이 좁은 인터포저에 밀집 → 인접 결합 폭증. 차폐·간격·차동의 한계.
3. **타이밍 정렬(skew) 폭증** — §7-3. 2048비트가 **같은 시각에** 도착해야 하는데, 배선 길이·TSV 깊이(어느 층 die냐)에 따라 도착 시각이 다 다르다. die별·비트별 skew 보정(per-bit deskew training)이 막대.
4. **열·전압 변동** — 03 A8 · 03 B6 . 적층 발열로 가운데 die가 뜨겁고, 온도·IR drop이 die마다 달라 ZQ/DLL/DCC 보정 부담↑. 같은 신호도 die 위치마다 마진이 다름.
5. **테스트 접근성** — 적층 후 내부 die의 PHY를 외부에서 직접 못 본다 → MBIST·내부 루프백·DFT로 검증(03 c7 과 연결).

B_hbm.md Q1 브릿지의 결론이 여기서 완성된다: HBM은 "폭을 넓혀 per-pin SI를 쉽게 만든 대신, PI·타이밍 정렬·테스트로 난도를 옮긴" 구조다. 그래서 HBM4 base die는 **고속 I/O PHY + PI + 타이밍 + DFT가 집약된 디지털 로직 SoC**가 되고(B_hbm.md Q3 브릿지), 이것이 base die를 로직 파운드리 공정으로 가져가는 이유 중 하나다.

강점 브릿지: 03 A8 브릿지대로 — **적층·대역폭 폭증 = SI·타이밍·테스트 복잡도 폭증**. STA·CDC·DFT·검증 자동화가 정확히 이 병목을 겨냥한다. 2048비트 deskew training의 타이밍 sign-off, base die 다중 도메인 CDC, 적층 PHY MBIST — 전부 지원자가 하던 일의 메모리판이다.

10. 면접 연결: 30초 / 1분 골격 + 강점 브릿지

03 의 ★기출 "신호무결성 대응", 그리고 빈출 "DLL vs PLL", "DFE/CTLE 왜", "ODT/termination", "setup/hold와 고속 I/O"를 구술 골격으로.

10.1 ★ "신호무결성(SI)에 어떻게 대응하나?" (확인된 기출)

[30초] "SI는 신호가 채널을 지나며 깨지는 걸 막아 0/1을 제 시각에 정확히 받게 하는 겁니다. 고속에서 신호를 깨는 건 임피던스 부정합으로 인한 **반사**, 인접선의 **crosstalk**, 채널 대역 제한으로 인한 **ISI**, 그리고 **jitter**입니다. 대응은 임피던스 매칭·**ODT termination**으로 반사를 잡고, **차동 신호**와 간격·차폐로 crosstalk를 줄이고, **등화 (FFE/CTLE/DFE)**로 ISI를 보상합니다. 이 모든 걸 **아이 다이어그램**으로 보면 마진이 보입니다 — 아이의 가로폭이 timing 마진, 세로높이가 전압 마진입니다."

[1분 확장] 위 + "특히 PI(전력무결성)가 짝입니다. 광폭 고속 I/O는 수천 핀이 동시에 스위칭해 **SSN(동시 스위칭 노이즈)**과 **IR drop**으로 전원이 흔들리고, 그게 곧 신호·클럭 jitter로 번집니다. 그래서 **PDN 설계**와 **decap**으로 전원을 받쳐줍니다. HBM4의 2048-bit는 per-pin 속도는 낮춰 SI를 쉽게 했지만, 동시 스위칭 핀 수가 2배라 PI와 타이밍 정렬이 폭증하는 구조입니다."

[강점 브릿지] "제가 STA에서 보던 setup/hold 마진은 시간축 마진이고, 고속 I/O 아이는 거기에 전압축 마진을 더한 겁니다. **마진을 sign-off로 지킨다는 사고가 동일**해서, timing closure 경험을 HBM/DDR I/O 마진 검증으로 바로 이식할 수 있습니다."

10.2 ★ "DLL과 PLL의 차이는?" (빈출 C9)

[30초] "둘 다 클럭을 잠그는 피드백 회로지만, **PLL은 VCO(발진기)가 있어 주파수 합성(체배)이 되고, DLL은 지연선만 있어 위상 정렬(skew 제거)만 합니다**. DLL은 발진기가 없어 입력 jitter를 누적·증폭하지 않아 안정적입니다. 그래서 DRAM은 **내부 지연으로 들어진 출력 DQ/DQS를 외부 클럭에 다시 정렬**할 때, 주파수는 그대로 두고 위상만 맞추면 되니 DLL이 잘 맞습니다. 주파수 합성이 필요하면 PLL을 씁니다."

[브릿지] "STA에서 jitter를 clock uncertainty로 보던 게, 회로로 내려오면 'PLL이 jitter를 만드냐 DLL이 안 만드냐'의 선택 문제가 됩니다. clock 품질이 곧 고속 I/O 아이 폭이라, 제 timing 사고와 직결됩니다."

10.3 "DFE를 왜, CTLE와 어떻게 다른가?"

[30초] "둘 다 채널 손실로 생긴 ISI를 보상하는 등화기인데, **CTLE는 RX 앞단의 아날로그 고역 부스트라 신호도 노이즈도 같이 증폭**합니다. **DFE는 이미 판정한 과거 비트로 그 잔향만 계산해 빼므로 노이즈를 안 키우고 ISI만 정확히 제거**합니다. 그래서 멀티드롭·반사가 큰 DDR 채널엔 DFE가 효율적이라 **DDR5가 DFE를 도입**했습니다. 단 DFE는 1 UI 안에 판정→감산 루프가 닫혀야 해서 타이밍이 빡빡합니다."

[브릿지] "그 DFE 피드백 루프는 결국 1 UI 안에 닫혀야 하는 **timing-critical path**라, setup 닫힘 문제와 같은 STA 사고로 뵙니다."

10.4 "ODT/termination이 왜 필요한가?"

[30초] "임피던스가 다른 경계에서 신호가 반사돼 ringing·ISI를 만듭니다($\Gamma = (Z_L - Z_0) / (Z_L + Z_0)$). 이걸 막으려면 채널 끝 임피던스를 채널 특성 임피던스와 맞춰 반사 에너지를 흡수해야 하고, 그 종단 저항을 칩 안에 넣은 게 **ODT**입니다. series는 source에서 흡수(저전력·점대점), parallel은 라인 끝에서 흡수(고속·멀티드롭)입니다. 그런데 온칩 임피던스가 **PVT로 흔들리니 ZQ 캘리브레이션**으로 외부 정밀 기준에 맞춰 코드로 보정합니다."

[브릿지] "ZQ는 PVT 변동을 런타임 보정 코드로 잡는 피드백 루프라, STA에서 PVT corner를 보던 사고가 calibration으로 진화한 형태입니다 — 게다가 그 보정·ODT 제어 FSM은 Peri 디지털 로직, 제 검증 대상입니다."

10.5 "setup/hold 마진과 고속 I/O 마진의 관계는?"

[30초] "본질이 같습니다. STA의 setup/hold는 데이터가 클럭 에지 기준 안전 윈도우에 들어왔나를 보는 **시간축 마진**입니다. 고속 I/O 아이 다이어그램은 거기에 **전압축(eye height)**을 더한 거고요. DDR은 **DQS로 DQ를 캡처**하니, DQ가 DQS 대비 data eye 안에 들어오게 read/write **leveling·training**으로 맞춥니다 — 이게 곧 setup/hold 마진을 실리콘에서 능동 조정하는 겁니다."

[총괄 브릿지 — 외워둘 한 문장] "제 4년은 로직의 timing closure·CDC·검증 sign-off였습니다. 고속 메모리 I/O의 마진 sign-off는 **같은 마진 사고에 전압축과 아날로그 변동(PVT calibration)**을 더한 것이라, STA·CDC·DFT 역량이 HBM/DDR PHY 마진 검증으로 1:1 이식됩니다. 모르는 건 디바이스 물리지, **마진을 지키는 방법론은 제가 매일 하던 일**입니다."

11. 스스로 점검 (정독 후 막힘없이 답해보라)

1. 대역폭 식 $\text{폭} \times \text{속도} \div 8$ 의 두 손잡이를 올릴 때 각각 어떤 무결성 문제(SI/PI)가 폭증하나? GDDR과 HBM은 어느 손잡이를 택했나?
2. ISI가 "데이터 패턴 의존적"인 이유를 채널이 LPF라는 사실로부터 설명하라. 그것이 아이를 어떻게 단나?
3. 아이 다이어그램의 가로폭·세로높이가 각각 어떤 마진이고, 어떤 열화(jitter/노이즈/반사)가 어느 쪽을 갇나? STA의 setup/hold 윈도우와 어떻게 대응되나?
4. FFE·CTLE·DFE를 위치(TX/RX)·방식·노이즈 증폭 여부로 비교하라. DDR5가 DFE를 고른 이유를 한 문장으로.
5. DLL과 PLL의 핵심 소자·잘하는 일·jitter 특성 차이는? DRAM 출력 정렬에 왜 DLL이 맞나?
6. ZQ 캘리브레이션이 "설계 때 고정값"이 아니라 런타임 보정이어야 하는 이유(1st-principles)는? STA의 PVT corner와 어떻게 연결되나?
7. series vs parallel termination의 위치·전력·용도 차이를 말하고, 고속 멀티드롭에 어느 것이 유리한지 이유와 함께.
8. PAM4가 NRZ 대비 얻는 것과 잃는 것은? eye height가 왜 약 1/3로 줄고, 그래서 무엇이 추가로 필요한가?
9. 광폭 고속 I/O에서 PI(SSN/IR drop)가 SI만큼 중요한 이유를 di/dt와 동시 스위칭 핀 수로 설명하라. PI 열화가 어떻게 SI(eye)로 번지나?
10. HBM 2048-bit에서 SI/PI/타이밍이 폭증하는 4가지 메커니즘을 들고, "폭을 넓혀 per-pin SI를 쉽게 한 대신 난도를 어디로 옮겼나"를 한 문장으로 정리하라.
11. (강점 통합) 본인의 STA·CDC·DFT·검증 sign-off 경험을, 고속 메모리 PHY 마진 검증으로 잇는 브릿지 문장을 30초로 말해보라.

⚠ 정확성 노트: 본문의 구체 수치(per-pin 속도, 종단 저항 Ω , σ 배수, dB, 노드 등)는 **세대·제품·문서별로 변동**한다. 면접에서는 "원리와 방향성"(예: "PAM4는 eye를 1/3로 깎는 대신 비트를 2배 씌는다", "DFE는 노이즈를 안 키우고 ISI만 뺀다")을 정확히 말하고, 구체 숫자는 "약/급/변동" 단서를 붙여 단정하지 말 것. 아날로그 회로 내부 세부보다 **"디지털 설계자가 이해해야 할 마진 사고"** 우선.